Informe TÃ©cnico: Arquitectura de ConexiÃ³n entre Microsoft SQL Server y Java en el MÃ³dulo de Inicio de SesiÃ³n

AnÃ¡lisis de Conectividad mediante Java Database Connectivity, Seguridad contra InyecciÃ³n SQL y Trazabilidad de AuditorÃ-a

Alessandro Villogas Gaspar

Escuela Profesional de IngenierÃ-a de Sistemas y ComputaciÃ³n

Facultad de IngenierÃ-a, Universidad Peruana Los Andes

Desarrollo de Aplicaciones I

Mg. Ing. RaÃºl FernÃ¡ndez Bejarano

Octubre de 2026

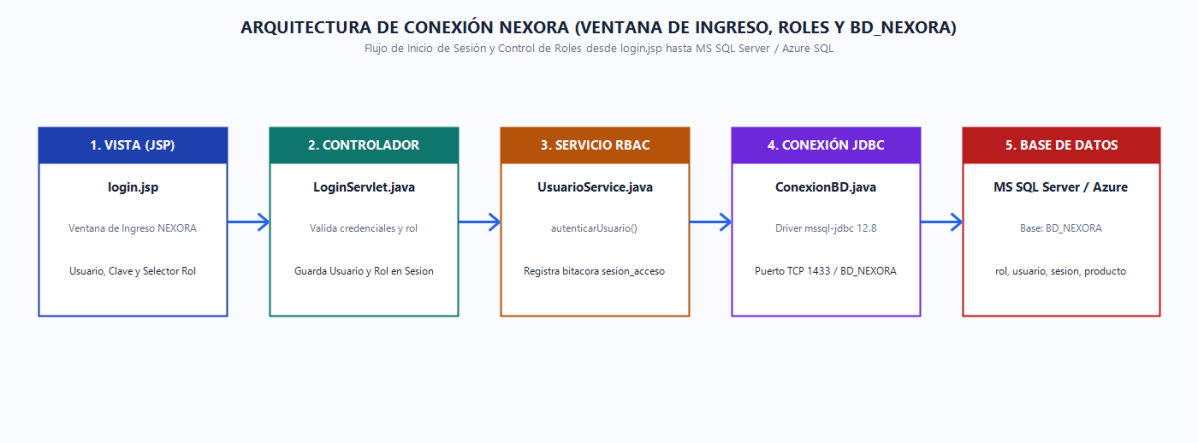
IntroducciÃ³n y FundamentaciÃ³n de la Conectividad

El acceso seguro a los sistemas de informaciÃ³n contemporÃ¡neos exige una separaciÃ³n nÃ-tida entre la capa de presentaciÃ³n que interactÃºa con el usuario y el motor de almacenamiento donde reposan los datos confidenciales. En la plataforma tecnolÃ³gica comercial NEXORA, la ventana de ingreso cumple el rol primordial de autenticar a los operadores y clientes, determinando de manera autÃ³noma los privilegios correspondientes a cada perfil sin requerir selecciÃ³n manual.

La vinculaciÃ³n entre la mÃ¡quina virtual de Java y el motor de bases de datos Microsoft SQL Server se realiza a travÃ©s de la interfaz Java Database Connectivity, empleando el controlador oficial de Microsoft. Este mecanismo permite enviar instrucciones estructuradas mediante el protocolo de red TCP sobre el puerto estandarizado 1433, asegurando una comunicaciÃ³n cifrada y de alto rendimiento.

Figura 1

Flujo de ComunicaciÃ³n Multicapa para la AutenticaciÃ³n de Usuarios y DetecciÃ³n AutomÃ¡tica de Roles



*Nota.*InteraccÃ³n desde la vista de ingreso hasta el almacenamiento persistente en SQL Server con resoluciÃ³n autÃ³noma del perfil.

EspecificaciÃ³n de la Cadena de ConexiÃ³n y ConfiguraciÃ³n de Red

La comunicaciÃ³n fÃ-sica depende de una cadena de conexiÃ³n unificada que define el destino de red, el nombre de la base de datos, el canal seguro y los parÃ¡metros de tolerancia a fallos. La instrucciÃ³n establecida en la clase ConexionBD.java responde a la siguiente estructura formal:

```
jdbc:sqlserver://localhost:1433;databaseName=BD_NEXORA;encrypt=true;trustServerCe
rtificate=true;loginTimeout=2;
```

• **Controlador Oficial:** Se hace uso de la clase com.microsoft.sqlserver.jdbc.SQLServerDriver provista por la biblioteca mssql-jdbc en su versiÃ³n doce.

• **Destino y Puerto de Red:** La direcciÃ³n localhost y el puerto 1433 dirigen el trÃ¡fico al motor relacional local o hacia la nube en Azure.

• **Cifrado del Canal:** Los parÃ¡metros encrypt y trustServerCertificate garantizan que las credenciales no viajen en texto plano por la red.

• **Tiempo LÃ-mite de Espera:** La propiedad loginTimeout configurada en dos segundos previene bloqueos indefinidos en el servidor web si la base de datos estuviera apagada.

Estructura de Base de Datos Vinculada a la Seguridad

El esquema de seguridad en la base de datos BD_NEXORA se sustenta en tres tablas relacionales perfectamente coordinadas para evitar duplicidades y mantener la trazabilidad de cada evento.

Tabla 1

Tablas del Esquema de Seguridad y Control de Acceso

Entidad FÃ-sica	Campos Principales	Responsabilidad en el Proceso de Ingreso
dbo.rol	Identificador de rol, nombre, descripciÃ³n y nivel de acceso	Define los perfiles de Administrador, AlmacÃ©n, Cliente y Consulta
dbo.usuario	Identificador, nombre de cuenta, contraseÃ±a, nombre completo, correo y estado activo	Valida las credenciales ingresadas en el formulario web
dbo.sesion_acceso	Identificador de sesiÃ³n, usuario vinculado, fecha de ingreso, direcciÃ³n de red y Ã©xito	Guarda el registro de auditorÃ-a de cada intento de conexiÃ³n

*Nota.*Tablas relacionales que intervienen en la autenticaciÃ³n del sistema NEXORA.

Recorrido Paso a Paso del Proceso de AutenticaciÃ³n

• **Primer paso, captura en la vista:** El usuario ingresa Ãºnicamente su nombre de cuenta y contraseÃ±a en el formulario web del archivo login.jsp, prescindiendo de selectores manuales de rol y transmitiendo los datos mediante el mÃ©todo seguro POST.

• **Segundo paso, recepciÃ³n en el controlador:** El servlet LoginServlet recibe los datos y captura la direcciÃ³n de red remota mediante la instrucciÃ³n req.getRemoteAddr para fines de auditorÃ-a.

• **Tercer paso, apertura del canal de datos:** La clase utilitaria ConexionBD proporciona una conexiÃ³n activa mediante DriverManager.getConnection hacia Microsoft SQL Server.

• **Cuarto paso, consulta parametrizada y resoluciÃ³n de rol:** El servicio UsuarioService ejecuta una consulta estructurada uniendo las tablas dbo.usuario y dbo.rol, verificando que la cuenta estÃ© activa, validando la clave y recuperando dinÃ¡micamente el perfil del usuario.

• **Quinto paso, registro de trazabilidad:** Inmediatamente despuÃ©s de validar la identidad, se inserta una fila en la tabla dbo.sesion_acceso registrando la marca temporal del servidor y la direcciÃ³n de red del equipo solicitante.

• **Sexto paso, establecimiento de sesiÃ³n y redirecciÃ³n adaptativa:** El controlador invalida cualquier identificador de sesiÃ³n previo para prevenir la fijaciÃ³n de sesiÃ³n, crea una nueva sesiÃ³n segura y redirige al panel correspondiente: tienda tecnolÃ³gica para el cliente o panel de control operativo para el administrador y almacÃ©n.

Estrategias de Seguridad y Resiliencia Aplicadas

Para asegurar que la soluciÃ³n satisfaga estÃ¡ndares de calidad en software corporativo, se aplicaron las siguientes medidas tÃ©cnicas:

• **Inmunidad ante InyecciÃ³n SQL:** Se prohÃ-be terminantemente concatenar variables en las cadenas de consulta. Se utilizan exclusivamente sentencias preparadas con parÃ¡metros tipados que neutralizan cualquier intento de manipulaciÃ³n sintÃ¡ctica.

• **GestiÃ³n Concurrente de Recursos:** Todas las conexiones y cursores se gestionan bajo bloques con cierre automÃ¡tico para liberar sockets y evitar fugas de memoria en el servidor.

• **Mecanismo de Resiliencia y Continuidad:** Si la base de datos externa no se encuentra en lÃ-nea, el sistema conmuta de forma transparente a una colecciÃ³n en memoria, asegurando que la evaluaciÃ³n y uso de la plataforma nunca se interrumpa.